

Unit 2

Joins: inner, outer (left/right/full), self-joins

SQL JOIN

A **JOIN** clause is used to combine rows from two or more tables, based on a related column between them.

Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

Then, look at a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Country
1	Alfreds Futterkiste	Maria Anders	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Mexico

3	Antonio Moreno Taquería	Antonio Moreno	Mexico
---	-------------------------	----------------	--------

Notice that the "CustomerID" column in the "Orders" table refers to the "CustomerID" in the "Customers" table. The relationship between the two tables above is the "CustomerID" column.

Then, we can create the following SQL statement (that contains an **INNER JOIN**), that selects records that have matching values in both tables:

Example [Get your own SQL Server](#)

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers ON Orders.CustomerID=Customers.CustomerID;
```

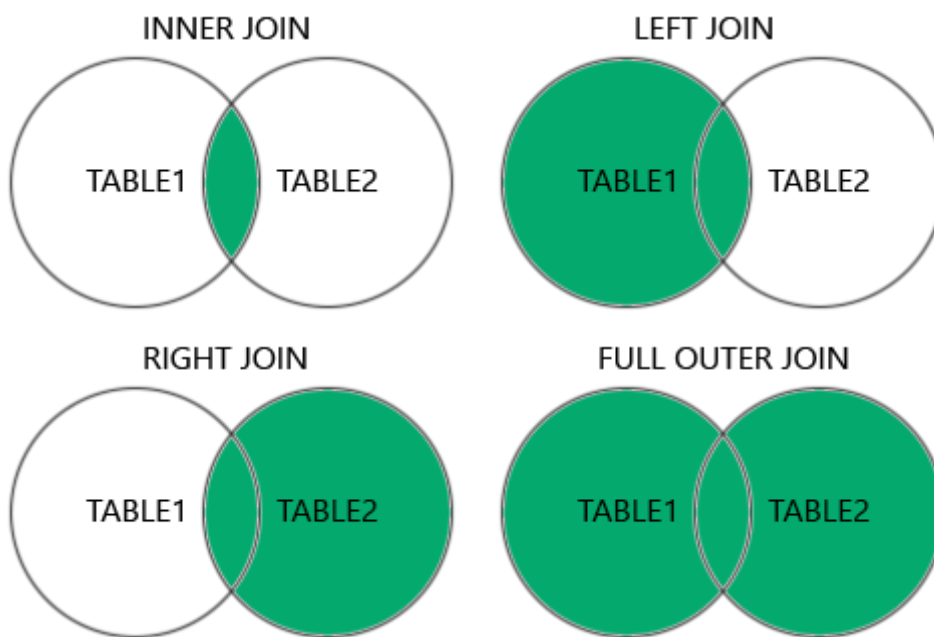
and it will produce something like this:

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	9/18/1996
10365	Antonio Moreno Taquería	11/27/1996
10383	Around the Horn	12/16/1996
10355	Around the Horn	11/15/1996
10278	Berglunds snabbköp	8/12/1996

Different Types of SQL JOINS

Here are the different types of the JOINS in SQL:

- **(INNER) JOIN**: Returns records that have matching values in both tables
- **LEFT (OUTER) JOIN**: Returns all records from the left table, and the matched records from the right table
- **RIGHT (OUTER) JOIN**: Returns all records from the right table, and the matched records from the left table
- **FULL (OUTER) JOIN**: Returns all records when there is a match in either left or right table



Indexes, sequences, and triggers in databases

Database systems utilize several key components, including indexes, sequences, and triggers, to optimize performance, manage data integrity, and automate tasks.

1. Indexes

Purpose: Indexes are special lookup tables that a database search engine can use to speed up data retrieval operations.

How it works: Imagine searching for a specific word in a book without an index – you'd have to read every page. A database index is like a book index: it creates a sorted list of values from one or more columns along with pointers (locations) to the actual data rows within the table. This allows the database to quickly find specific records without scanning the entire table.

Types: Common index types include:

Clustered Indexes: Determines the physical order of data in the table itself. Only one clustered index is allowed per table.

Non-clustered Indexes: Doesn't change the physical order of data, but stores the indexed values and pointers to the actual data locations separately. A table can have multiple non-clustered indexes.

Bitmap Indexes: Efficient for columns with low cardinality (few distinct values).

Hash Indexes: Provides very fast exact-match lookups but less efficient for range queries.

Advantages:

Faster data retrieval.

Improved query performance.

Can be used for sorting, eliminating post-fetch sorting.

Unique indexes ensure data integrity by preventing duplicate values.

Disadvantages:

Require additional storage space.

Can decrease performance on data modification operations (inserts, updates, deletes) because the index needs to be updated as well.

Choosing the right indexing strategy can be complex.

2. Sequences

Purpose: Sequences are database objects designed to automatically generate unique numerical values, according to GeeksforGeeks. These values are commonly used for primary keys and other unique identifiers in databases.

How it works: You define a sequence with a starting value and an increment value. Each time the sequence is called, it returns the next available unique number. Sequences can be configured to generate numbers in ascending or descending order and can even cycle back to the beginning once the maximum value is reached.

Advantages:

Simplified unique value generation, removing the need for complex application logic.

Ensures uniqueness, which is crucial for primary keys.

Independent of tables, allowing a single sequence to be used across multiple tables.

Can be cached in memory for faster generation.

Disadvantages:

Cached sequences can experience gaps in numbering if the system restarts.

Not supported by all database management systems. For example, MySQL uses the `AUTO_INCREMENT` attribute instead of explicit sequences.

3. Triggers

Purpose: Triggers are predefined actions that a database automatically executes in response to certain events occurring on a table or view.

How it works: A trigger is attached to a specific database event (like INSERT, UPDATE, or DELETE) and a target table or view. When that event occurs, the database automatically executes the trigger's code (a set of SQL statements). Triggers can be set to execute either before or after the event.

Types:

BEFORE Triggers: Execute before the triggering event and can be used for data validation or modifying data before it's inserted, updated, or deleted.

AFTER Triggers: Execute after the triggering event and are suitable for tasks like auditing changes or updating related tables.

INSTEAD OF Triggers: Replace the triggering event with a custom action, often used with views to perform complex operations that standard DML statements can't handle directly.

Row-Level Triggers: Execute for each row affected by the triggering event.

Statement-Level Triggers: Execute only once per SQL statement, regardless of the number of rows affected.

Advantages:

Automates repetitive tasks, reducing manual intervention and ensuring consistency.

Enforces business rules and maintains data integrity by executing predefined actions during data modifications.

Facilitates auditing and logging by automatically recording changes to data.

Disadvantages:

Can introduce complexity and be difficult to debug.

Can negatively impact database performance if poorly designed or if they contain complex logic.

Can potentially lead to unintended consequences if not carefully managed (e.g., cascading triggers).